OOP W22 MODEL ANSWER MSBTE

This Study Material is provided by Campusifyplus.in ©

Made by Team Campusify

THIS IS THE FREE STUDY MATERIAL BY CAMPUSIFY!!

Be Part of Our Community

Telegram Channel - https://t.me/Campusifyy

Whatsapp Channel - https://tinyurl.com/Campusify-channel

Campusify is building the Biggest Diploma/Engineering Student Community in Maharashtra. Be part of the movement!

Join Channel for MSBTE Study Materials



Earn by Referring Campusify Course

Presented by **Campusify** | **10K** Active Members | **2.7L** Students Helped within **1 year**

Refer Our course with your **Third year** friend or cousin and get **10%** commission on each referral.

For More Details : WhatsApp Us at **+91 8291934640**

TY Course Link - https://www.campusifyplus.site

WINTER – 2022 EXAMINATION
MODEL ANSWER

Subject: Object Oriented Programming with C++ **Subject Code** **22316**

Important Instructions to examiners:

1) The answers should be examined by key words and not as word-to-word as given in the model answer scheme.
2) The model answer and the answer written by candidate may vary but the examiner may try to assess the understanding level of the candidate.
3) The language errors such as grammatical, spelling errors should not be given more Importance (Not applicable for subject English and Communication Skills.
4) While assessing figures, examiner may give credit for principal components indicated in the figure. The figures drawn by candidate and model answer may vary. The examiner may give credit for anyequivalent figure drawn.
5) Credits may be given step wise for numerical problems. In some cases, the assumed constant values may vary and there may be some difference in the candidate's answers and model answer.
6) In case of some questions credit may be given by judgement on part of examiner of relevant answer based on candidate's understanding.
7) For programming language papers, credit may be given to any other program based on equivalent concept.
8) As per the policy decision of Maharashtra State Government, teaching in English/Marathi and Bilingual (English + Marathi) medium is introduced at first year of AICTE diploma Programme from academic year 2021-2022. Hence if the students in first year (first and second semesters) write answers in Marathi or bilingual language (English +Marathi), the Examiner shall consider the same and assess the answer based on matching of concepts with model answer.

Q. No	Sub Q.N.	Answer	Marking Scheme
1.	**a) Ans.**	**Attempt any <u>FIVE</u> of the following:** **State any four applications of OOP** Following are the application of OOP: • Most popular application of OOP is in the area of user interface design such as windows. Hundreds of windowing systems have been developed using OOP techniques. • C++ is suitable for any programming task including development of editors, compilers • Developing databases • Developing communication systems and complex real-time applications. • Simulation and modeling. • AI and Expert system	**10** **2M** ***Any four applications ½M each***

WINTER – 2022 EXAMINATION
MODEL ANSWER

Subject: Object Oriented Programming with C++ **Subject Code** **22316**

	b) Ans.	**Explain user defined data types with example** The data types that are defined by the user are called user-defined data type. **Examples:** 1. Class 2. Structure 3. Union 4. Enumeration	**2M** ***Definition 1M*** ***Two examples 1M***

	c) Ans.	**Describe use of scope resolution operator with example.**
		• Scope of a variable extends from the point of declaration to the end of the block.
		• A variable declared inside a block is 'local' variable and a variable declared outside block is called as global variable.
		• When we want to use a global variable but also has a local variable with same name.
		• To access the global version of the variable, C++ provides scope resolution operator.
		#include <iostream>
		int n = 12; // A global variable
		int main()
		{
		int n = 13; // A local variable
		cout<< ::n<<endl;
		// Print the global variable: 12
		cout<< n <<endl;
		// Print the local variable: 13
		}
			2M
			Explanation 1M
			Example 1M

	d) Ans.	**Define constructor's and its type** A constructor is a special member function that is executed automatically whenever the object of a class is created. Following are the types of constructors: 1. Default constructor 2. Parameterized constructor 3. Copy constructor 4. Constructor with default argument	**2M** ***Definition 1M any two types 1M***
	e) Ans.	**Explain concept of pointer with example** A pointer is a variable that holds an address of another variable. They have data type just like variables, for example an integer type pointer can hold the address of an integer variable and a character type	**2M** ***Concept / Definition 1M***

pointer can hold the address of char variable.
Example :

```
int *ptr;          // ptr is a pointer
int a = 10;        //'a' is ordinary variable holding an integer
ptr = &a;          //initializing pointer with the address of 'a'
```

Example 1M

f) Ans. **Explain file modes used to perform file operations.** **2M**

Following are the file modes used to perform file operations

Any two file modes 1M each

Parameter	Meaning
ios :: app	Append to end-of-file
ios :: ate	Go to end-of-file on opening
ios :: binary	Binary file
ios :: in	Open file for read only
ios :: nocreate	Open fails if file does not exist
ios :: noreplace	Open fails if file does already exist
ios :: out	Open file for writing only
ios :: trunc	Delete the contents of the file if it exists

g) Ans. **List all stream classes used in stream operation** **2M**

Following are the stream classes used in stream operation

Listing of Any four stream classes ½M each

- ios
- istream
- ostream
- iostream
- streambuf
- filebuf
- fstreambase
- ifstream
- ofstream
- fstream

2. **Attempt any <u>THREE</u> of the following:** **12**

a) Ans. **Develop a C++ program to print Fibonacci series** **4M**

Correct logic 2M

```
#include <iostream>
using namespace std;
int main()
```

		{
		int t1 = 0, t2 = 1, nextTerm = 0, n,count;
		cout<< "Enter a positive number: ";
		cin>> n;
		// displays the first two terms which is always 0 and 1
		cout<< "Fibonacci Series: " << t1 << " " << t2 <<" ";
		count=2;
		while(count!= n)
		{
		nextTerm = t1 + t2;
		cout<<nextTerm<<" ";
		t1 = t2;
		t2 = nextTerm;
		count++;
		}
		return 0;
		}
			Correct syntax 2M

	b) Ans.	**Develop a c++ program for multilevel inheritance.**
		(Any program with correct logic of multilevel inheritance shall be considered)
		#include <iostream>
		using namespace std;
		class base //single base class
		{
		public:
		int x;
		void getdata()
		{
		cout<< "Enter value of x= "; cin>> x;
		}
		};
		class derive1 : public base // derived class from base class
		{
		public:
		int y;
		void readdata()
		{
		cout<< "\nEnter value of y= "; cin>> y;
		}
			4M
			
			Correct logic 2M
			
			Correct syntax 2M

```
};
class derive2 : public derive1   // derived from class derive1
{
        private:
        int z;
        public:
        void indata()
        {
        cout<< "\nEnter value of z= "; cin>> z;
        }
        void product()
        {
        cout<< "\nProduct= " << x * y * z;
        }
};
int main()
{
    derive2 a;      //object of derived class
a.getdata();
a.readdata();
a.indata();
a.product();
    return 0;
}
```

c) **Develop a c++ program for accept data from user to calculate percentage for 5 subjects and display grade according to percentage.** **4M**

Ans.

Correct logic 2M

Correct syntax 2M

```
#include<iostream>
using namespace std;
int main()
{
        int phy,chem,bio,maths,comp;
        int total;
        float percent;
        cout<<"Enter marks of computer";
        cin>>phy;
        cout<<"enter marks of English";
        cin>>chem;
        cout<<"enter marks of bio";
```

```
        cin>>bio;
        cout<<"enter marks of maths";
        cin>>maths;
        cout<<"enter marks of physics"; cin>>comp;
        total= phy+chem+bio+maths+comp;
        percent=total/5.0;
        if(percent>=90)
        {
                cout<<"Grade=A+";
        }
        else if(percent>=80)
        {
                cout<<"Grade=A";
        }
        else if(percent>=70)
        {
                cout<<"Grade=B+";
        }
else if(percent>=60)
        {
                cout<<"Grade=B";
        }
        else if(percent>=40)
        {
                cout<<"Grade=C";
        }
        else
        {
                cout<<"Grade= Fail";
        }
}
```

d) **Develop c++ program to open and read content of file also write "object oriented" string in fi1e and close it'** **4M**

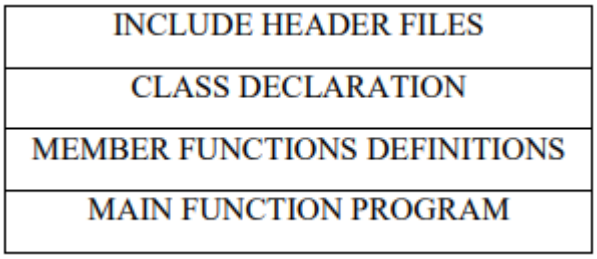
Ans.

```
#include<iostream>
#include<fstream>
using namespace std;
int main()
{
```

Correct logic 2M

Correct syntax 2M

		ofstreamfout;
		fout.open("abc.txt",ios::out);
		fout<<"Object Oriented";
		fout.close();
		ifstream fin;
		char line[100];
		fin.open("abc.txt",ios::in);
		cout<<"Contents of file are:\n";
		while(fin)
		{
		fin.getline(line,100);
		cout<<line<<"\n";
		}
		fin.close();
		return 0;
		}	

3.	**a)** **Ans.**	**Attempt any <u>THREE</u> of the following:** **Describe structure of C++ program** General C++ program has following structure.  Description: - 1. Include header files In this section a programmer include all header files which are require to execute given program. The most important file is iostream.h header file. This file defines most of the C++statements like cout and cin. Without this file one cannot load C++ program. 2. Class Declaration In this section a programmer declares all classes which are necessary for given program. The programmer uses general syntax of creating class. 3. Member Functions Definition This section allows programmer to design member functions of a	**12** **4M** ***Correct diagram 2M*** ***Description 2M***

		class. The programmer can have inside declaration of a function or outside declaration of a function. 4. Main Function Program In this section programmer creates objects and calls various functions writer within various class.	

	b)
	Ans.
		Explain with suitable example Friend Function.
		Friend function:
		The private members of a class cannot be accessed from outside the class but in some situations two classes may need access of each other's private data. So a common function can be declared which can be made friend of more than one class to access the private data of more than one class. The common function is made friendly with all those classes whose private data need to be shared in that function. This common function is called as friend function. Friend function is not in the scope of the class in which it is declared. It is called without any object. The class members are accessed with the object name and dot membership operator inside the friend function. It accepts objects as arguments.
		Example:
		Program to interchange values of two integer numbers using friend function.
		#include<iostream.h>
		#include<conio.h>
		class B;
		class A
		{
		int x;
		public:
		void accept()
		{
		cout<<"\n Enter the value for x:";
		cin>>x;
		}
		friend void swap(A,B);
		};
		class B
		{
		int y;
		public:
			4M
			Correct explanation 2M
			Any Correct example 2M

```
  void accept()
  {
  cout<<"\n Enter the value for y:";
  cin>>y;
  }
  friend void swap(A,B);
  };
  void swap(A a,B b)
  {
  cout<<"\n Before swapping:";
  cout<<"\n Value for x="<<a.x;
  cout<<"\n Value for y="<<b.y;
  int temp;
  temp=a.x;
  a.x=b.y;
  b.y=temp;
  cout<<"\n After swapping:";
  cout<<"\n Value for x="<<a.x;
  cout<<"\n Value for y="<<b.y;
  }
  void main()
  {
  A a;
  B b;
  clrscr();
  a.accept();
  b.accept();
  swap(a,b);
  getch();
   }
```

	c) Ans.	**Describe all visibility modes and effects with example.** Visibility modes used in inheritance are: • Private • Protected • Public	**4M** ***List of Visibility modes 1M*** ***Explanation of each mode 2M,*** ***Example 1M each***

Base class visibility	Derived class visibility		
	Private	Protected	Public
Private	Not Inherited	Not Inherited	Not Inherited
Protected	Private	Protected	Protected
Public	Private	Protected	Public

Fig: Visibility modes in inheritance

Private:

When a base class is privately inherited by a derived class, public members and protected members of the base class become private members of the derived class.

Therefore, the public and protected members of the base class can only be accessed by the member functions of derived class but, cannot be accessed by the objects of the derived class.

Example:

```
class Result: private Student
{
float percentage;
};
```

Public:

When a base class is publicly inherited by a derived class then protected members of base class becomes protected members and public members of the base class become public members of the derived class.

Therefore, the public members of the base class can be accessed by both the member functions of derived class as well as the objects of the derived class.

Example:

```
class Result: public Student
{
float percentage;
};
```

Protected:
When a base class is inherited by derived class in protected visibility mode, public and protected members of the base class become protected members of the derived class.
Therefore the public and protected members of the base class can be accessed by the member functions of derived class as well as the member functions of immediate derived class of it but they cannot be accessed by the objects of derived class.

Example:

```
class Result: protected Student
{
float percentage;
};
```

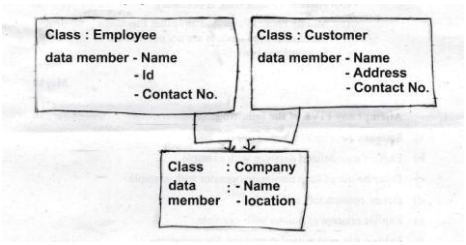
d) Ans. **Differentiate between Compile time polymorphism and Runtime polymorphism** — **4M**

Any four differences 1M each

Sr. No.	Compile time polymorphism	Runtime polymorphism
1	The linking of function call to function definition is done at compile time.	The linking of function call to function definition is done at run time.
2	Functions to be executed are known well before(at compile time)	Function to be executed is unknown until appropriate selection is made at run time
3	This does not require use of pointers to objects	This requires use of pointers to object
4	Function calls execution are faster	Function calls execution are slower
5	It is implemented with operator overloading or function overloading	It is implemented with virtual function.

4. **Attempt any THREE of the following:** — **12**

a) **Develop a c++ program to implement inheritance shown in following fig** — **4M**

	Ans.	
		#include <iostream>
		using namespace std;
		class Employee
		{
		protected:
		char Name[20];
		int Id;
		long contact_noE;
		};
		class Customer
		{
		protected:
		char Name[20];
		char Address[50];
		long contact_noC;
		};
		class Company:publicEmployee,public Customer
		{
		char Name[20];
		char location[20];
		public:
		void getE()
		{
		cout<<"\nEnter employee's data:";
		cout<<"\nName:";
		cin>>Name;
		cout<<"\nId:";
		cin>>Id;
		cout<<"\nContact no:";
		cin>>contact_noE;
		cout<<"\nCompany name:";
		cin>>Name;
			Correct definition of class Employee 1M,
			
			Correct definition of class Customer 1M,
			
			Correct definition of class Company 1M,
			
			Correct definition of main() 1M

```
cout<<"\nLocation:";
cin>>location;
   }
   void getC()
   {
cout<<"\nEnter customer's data:";
cout<<"\nName:";
cin>>Name;
cout<<"\nAddress:";
cin>>Address;
cout<<"\nContact no:";
cin>>contact_noC;
cout<<"\nCompany name:";
cin>>Name;
cout<<"\nLocation:";
cin>>location;
   }
   void putE()
   {
cout<<"\nEmployee's data is:";
cout<<"\nName:"<<Name;
cout<<"Id:"<<Id;
cout<<"\nContact no:"<<contact_noE;
cout<<"\nCompany name:"<<Name;
cout<<"\nLocation:"<<location;
   }
   void putC()
   {
cout<<"\nCustomer's data is:";
cout<<"\nName:"<<Name;
cout<<"Address:"<<Address;
cout<<"\nContact no:"<<contact_noC;
cout<<"\nCompany name:"<<Name;
cout<<"\nLocation:"<<location;
   }
};
intmain()
{
   Company C;
```

		C.getE(); C.getC(); C.putE(); C.putC(); return 0; }	

	b)
	Ans.
		Develop a c++ program to print sum of 10 nos. using array.
		#include<iostream.h>
		#include<conio.h>
		void main()
		{
		int arr[20],i,n,sum=0;
		clrscr();
		cout<<"\nEnter size of an array:";
		cin>>n;
		cout<<"\nEnter the elements of an array:";
		for(i=0;i<n;i++)
		{
		cin>>arr[i];
		}
		for(i=0;i<n;i++)
		{
		sum=sum+arr[i];
		}
		cout<<"\nArray elements are:";
		for(i=0;i<n;i++)
		{
		cout<<arr[i]<<" ";
		}
		cout<<"\nSum of array elements is:"<<sum;
		getch();
		}
			4M
			Initialization of array 2M,
			Logic of the program2M

	c) **Ans.**	**Develop a c++ program to perform arithmetic operation using pointer.** ***(Any other program with correct logic can be considered)*** #include<iostream.h> #include<conio.h> void main() {	**4M** ***Correct logic 2M*** ***Correct syntax 2M***

		int a=5,*ptr;
		clrscr();
		ptr=&a;
		cout<<a<<endl;
		cout<<*ptr<<endl;
		*ptr=*ptr+1;
		cout<<*ptr<<endl;
		*ptr=*ptr-1;
		cout<<*ptr<<endl;
		getch();
		}	

	d)
	
	Ans.
		Develop a c++ program to create structure student with data member Name, Roll No., percentage accept and display data for 5 student using structure.
		#include<iostream.h>
		#include<conio.h>
		struct student
		{
		char Name[20];
		int Roll_No;
		float percentage;
		}s[5];
		void main()
		{
		int i;
		clrscr();
		
		for(i=0;i<5;i++)
		{
		cout<<"Enter name of student "<<i+1;
		cin>>s[i].Name;
		cout<<"Enter Roll Number of student "<<i+1;
		cin>>s[i].Roll_No;
		cout<<"Enter percentage of student "<<i+1;
		cin>>s[i].percentage;
		}
		cout<<"\nStudents data is: ";
		for(i=0;i<5;i++)
		{
			4M
			
			Correct syntax for structure definition 2M
			
			Correct syntax for main function 2M

		cout<<"Student "<<i+1<<" Name: ";
		cout<<s[i].Name<<endl;
		cout<<"Student "<<i+1<<" Roll Number";
		cout<<s[i].Roll_No<<endl;
		cout<<"Student "<<i+1<<" Percentage: ";
		cout<<s[i].percentage<<endl;
		}
		getch();
		}	

	e)
	Ans.
		Develop c++ program to check Detection of end of file.
		(Any other relevant example shall be considered)
		#include<iostream>
		#include<fstream>
		using namespace std;
		int main()
		{
		ofstreamfout;
		fout.open("abc.txt",ios::out);
		fout<<"Object Oriented";
		fout.close();
		ifstream fin;
		char line[100];
		fin.open("abc.txt",ios::in);
		cout<<"Contents of file are:\n";
		char c;
		while(fin.get(c))
		cout<<c;
		
		if(fin.eof())
		cout<<"[EoFreached]\n";
		else
		cout<<"[error reading]\n";
		return 0;
		}
			4M
			
			Correct logic 2M
			
			Correct syntax 2M

5.	**a)**	**Attempt any <u>TWO</u> of the following:** **Define class book with following data member and member functions for 10 book.**	**12** **6M**

	Ans.	Data Member
		1. B - name
		2. B - author
		3. B - price
		Member Function
		→ getdata ( )
		→ put data ( )
		
		#include <iostream.h>
		class book
		{
		char name[20],author[20];
		int price;
		public:
		void getdata()
		{
		cout<<"Enter name";
		cin>>name;
		cout<<"Enter author";
		cin>>author;
		cout<<"Enter price";
		cin>>price;
		}
		void putdata()
		{
		cout<<endl<<"Name = "<<name;
		cout<<endl<<"Author = "<<author;
		cout<<endl<<"Price = "<<price;
		}
		};
		void main()
		{
		book b[10];
		int i;
		for(i=0;i<10;i++)
		b[i].getdata();
		for(i=0;i<10;i++)
		b[i].putdata();
		getch();
		}
			Declare class with data members and member functions- 4M
			
			Main function with function call in loop - 2M

	b) Ans.	**Develop a c++ program to implement virtual Base class.** ***(Any correct program with virtual base class shall be considered.)***	**6M** ***Program with grandparent class (virtual base class)- 1M,*** ***Parent class1-1M,*** ***Parent class2-1M,*** ***Child class- 2M,*** ***main function- 1M***

```
#include<iostream.h>
#include<conio.h>
class student
{
int rno;
public:
void getnumber()
{
cout<<"Enter Roll No:";
cin>>rno;
}
void putnumber()
{
cout<<"\n\n\t Roll No:"<<rno<<"\n";
}
};
class test: virtual public student
{
protected:
int part1,part2;

public:
void getmarks()
{
cout<<"Enter Marks\n";
cout<<"Part1:";
cin>>part1; cout<<"Part2:";
cin>>part2;
}
void putmarks()
{
cout<<"\t Marks Obtained\n";
cout<<"\n\t Part1:"<<part1;
cout<<"\n\tPart2:"<<part2;
}
};
class sports: public virtual student
```

```
{
protected:
int score;

public:
void getscore()
{
cout<<"Enter Sports Score:";
cin>>score;
}
void putscore()
{
cout<<"\n\t Sports Score is:"<<score;
}
};
class result: public test, public sports
{
int total;
public:
void display()
{
total=part1+part2+score;
putnumber();
putmarks();
putscore();
cout<<"\n\t Total Score:"<<total;
}
};
void main()
{
result obj;
clrscr();
obj.getnumber();
obj.getmarks();
obj.getscore();
obj.display();
getch();
}
```

	c)
	
	Ans.
		Explain rules of operator overloading and overload '+' operator to concatenate two strings.
		Rules for overloading operators:
		1. Only existing operators can be overloaded. New operators cannot be created.
		2. The overloaded operator must have at least one operand that is of user defined data type.
		3. We can't change the basic meaning of an operator. That is to say, we can't redefine the plus (+) operator to subtract one value from other.
		4. Overloaded operators follow the syntax rules of the original operators. They can't be overridden.
		5. There are some operators that can't be overloaded.
		6. We can't use friend functions to overload certain operators.
		7. Unary operators overloaded by means of member function take no explicit arguments and return no explicit values, but those overloaded by means of the friend function, take one reference argument (the object of the relevant class).
		8. Binary operators overloaded through a member function, take one explicit argument and those which are overloaded through a friend function take two explicit arguments.
		9. When using binary operators overloaded through a member function, the left-hand operand must be an object of the relevant class.
		10. Binary arithmetic operators such as +, -, * and / must explicitly return a value. They must not attempt to change their own arguments.
		
		Program:
		#include<string.h>
		class operatorplus
		{
		char str1[10];
		public:
		void getdata()
		{
		cout<<"\nEnter a string";
		cin>>str1;
		}
		void operator + (operatorplus s)
			6M
			Any four rules of operator overloading - ½ M each,
			
			program logic for overloading -2M,
			
			calling overloaded function 2M

		{
		cout<<strcat(str1, s.str1);
		}
		};
		void main ()
		{
		operatorplus op1, op2;
		clrscr();
		op1.getdata();
		op2.getdata();
		op1+op2;
		getch();
		}	

6.	**a)**
	
	Ans.
		Attempt any <u>TWO</u> of the following:
		Develop a c++ program for constructor with default argument and use of destructor
		(One program with both or two different programs shall be considered.)
		#include<iostream.h>
		#include<conio.h>
		#include<string.h>
		class student
		{
		int rno;
		char name[10];
		int year;
		public:
		student(int r,char n[],char y=1)
		{
		rno=r;
		strcpy(name,n);
		year=y;
		}
		void putdata()
		{
		cout<<rno;
		cout<<name;
		cout<<year;
		}
			12
			6M
			program for constructor with default argument- 3M
			
			use of destructor- 3M

```
~student()
{
cout<<"Destructor is executed:";
}
};
void main()
{
student s1(2,"abc");
student s2(3,"xyz",3);
clrscr();
s1.putdata();
s2.putdata();
getch();
}
```

OR

Program for use of destructor:

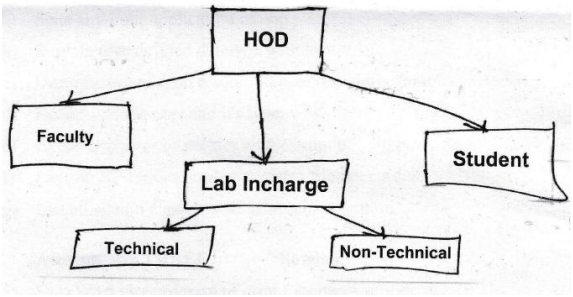
```
#include<iostream.h>
#include<conio.h>
int count=0;
class Test
{
   public:
Test()
      {
         count++;
cout<<"\n No. of Object created:\t"<<count;
      }

      ~Test()
      {
cout<<"\n No. of Object destroyed:\t"<<count;
         --count;
         }
};

void main()
{
```

		Test t,t1,t2,t3;
		getch();
		}	

	b)
	Ans.
		Describe Function overloading with suitable program
		Function overloading refers to the use of the same function name to create functions that perform a variety of different tasks. Each function has same name but different number of arguments or different types of arguments.
		The function is executed depending on the argument list in the function call.
		Program:
		#include<iostream.h>
		#include<conio.h>
		void swap(int a,int b)
		{
		int temp;
		temp=a;
		a=b;
		b=temp;
		cout<<"\nInteger values after swapping are:"<<a<<" "<<b;
		}
		void swap(float x,float y)
		{
		float temp1=x;
		x=y;
		y=temp1;
		cout<<"\nFloat values after swapping are:"<<x<<" "<<y;
		}
		void main()
		{
		clrscr();
		swap(10,20);
		swap(10.15f,20.25f);
		getch();
		}
		
		In the above program, swap( ) is overloaded by defining two different data type parameters in argument list. first swap( ) definition contains two integer parameters and second contains two float parameters.
			6M
			Description 3M,
			
			Any program with function overloading 3M

		when first function is called , compiler checks number of parameters and data type of each parameter with first function definition. If both are same then the definition executes and swapping of integer numbers is displayed. If the function call argument list does not match with first definition then second definition argument list is compared and if matches then swapping of float numbers is displayed.	
	c)	**Develop a c++ program to implement inheritance shown in fig** 	**6M**

	Ans.	***(Any other correct implementation shall be considered.)***
		#include<iostream.h>
		#include<conio.h>
		class HOD
		{
		protected:
		char name[10];
		char deptName[20];
		};
		class Faculty:public HOD
		{
		char fname[10];
		public:
		void getfaculty()
		{
		cout<<"Enter department name:";
		cin>>deptName;
		cout<<"Enter HOD name:";
		cin>>name;
		
		cout<<"Enter Faculty name:";
		cin>>fname;
			Each class implementation-1M

```
}
void putfaculty()
{
cout<<"\nDepartment name= "<<deptName;
cout<<"\nHOD name= "<<name;
cout<<"\nFaculty name= "<<fname;
}
};
class LabIncharge:public HOD
{
protected:
char lname[10];
};
class Technical:publicLabIncharge
{
char q[10];
public:
void getlabT()
{
cout<<"Enter department name:";
cin>>deptName;
cout<<"Enter HOD name:";
cin>>name;
cout<<"Enter Lab Incharge name:";
cin>>lname;
cout<<"Enter Qualification:";
cin>>q;
}
void putlabT()
{
cout<<"\nDepartment name= "<<deptName;
cout<<"HOD name= "<<name;
cout<<"Lab Incharge name: "<<lname;
cout<<"Qualification: "<<q;
}
};
class NonTechnical:publicLabIncharge
{
char q[10];
```

```
public:
void getlabN()
{
cout<<"Enter department name:";
cin>>deptName;
cout<<"Enter HOD name:";
cin>>name;
cout<<"Enter Lab Incharge name:";
cin>>lname;
cout<<"Enter Qualification:";
cin>>q;
}
void putlabN()
{
cout<<"\nDepartment name= "<<deptName;
cout<<"HOD name= "<<name;
cout<<"Lab Incharge name: "<<lname;
cout<<"Qualification: "<<q;
}};
class Student:public HOD
{
char sname[10];
public:
void getstudent()
{
cout<<"Enter department name:";
cin>>deptName;
cout<<"Enter HOD name:";
cin>>name;
cout<<"Enter Student name:";
cin>>sname;
}
void putstudent()
{
cout<<"\nDepartment name= "<<deptName;
cout<<"HOD= "<<name;
cout<<"Student name= "<<sname;
}};
void main(){
```

		Faculty f;
		Student s;
		Technical t;
		NonTechnical n;
		clrscr();
		f.getfaculty();
		s.getstudent();
		t.getlabT();
		n.getlabN();
		f.putfaculty();
		s.putstudent();
		t.putlabT();
		n.putlabN();
		getch();
		}	